

1 Abstract

This report details the statistical evaluation of temporal trends in High-Performance Liquid Chromatography (HPLC) detector signals and concentration metrics across 32 sequential chromatographic runs. The primary objective was to identify systematic deviations indicative of detector instability, column degradation, or sample matrix effects. Due to the dataset size (1.08 million rows) and data quality issues, including negative cumulative values and a 65% missing rate for concentration metrics, a rigorous data cleaning and pre-processing protocol was implemented. This involved capping negative values at zero, normalizing UV signals by system flow rate to isolate true drift from flow-induced artifacts, and excluding rows with missing concentration data where imputation was not feasible. The analysis aggregated metrics at the run level and stratified results by chromatography stage order to distinguish between gradual column fouling and sudden detector faults. Critical equipment failures were objectively defined using industry-standard thresholds, specifically a coefficient of determination (R^2) less than 0.99 for linear regression and CUSUM control limit exceedances. The results confirm that while most runs exhibited normal operational variance, specific stages demonstrated significant signal drift exceeding statistical limits, necessitating immediate maintenance intervention to ensure product quality and safety compliance.

2 Introduction

In the pharmaceutical and biotechnology sectors, the continuous monitoring of chromatographic processes is critical for maintaining product quality and ensuring safety. The integrity of HPLC data relies heavily on the stability of both the detector and the separation column over time. Systematic deviations in detector signals (UV_1_280_mAU) and concentration metrics (Conc_B_%) can arise from various sources, including gradual column degradation, sudden detector instability, or variations in sample matrix effects. Detecting these deviations early is essential for preventing the release of non-compliant products. This study focuses on the analysis of 32 sequential chromatographic runs, totaling 1.08 million data points. The primary motivation for this data analysis workflow was to quantify the rate and magnitude of signal drift over time, tracked via run number and chromatography stage order. A key challenge in this dataset was the presence of physically impossible negative values in cumulative variables and a high rate of missing data for concentration metrics. The analysis plan was designed to address these issues through strict data cleaning, normalization of signals against system flow rates to prevent pump variability from mimicking concentration drift, and the application of robust statistical thresholds (CUSUM and R^2) to differentiate between normal operational variance and critical equipment failures.

3 Methodology

The data analysis workflow consisted of three distinct phases: data cleaning and pre-processing, run-level aggregation and drift detection, and statistical threshold evaluation and failure classification. First, the raw data from 'chromatography_data.csv' underwent rigorous cleaning. Negative cumulative values for volume and UV signals were capped at zero. Missing concentration data (Conc_B_ml) was addressed by imputing missing values using the median of the chromatography stage; however, rows where imputation failed were excluded. The remaining 35% of concentration data, represented by Conc_B_%, was retained as the robust metric for analysis. Second, the cleaned data was aggregated at the 'run_no' level. Mean UV signals, concentration percentages, and system flow rates were calculated for each run. To isolate true drift from flow-induced artifacts, UV signals were normalized by dividing by the mean system flow rate. A rolling mean (window=15) was applied to the normalized UV data to detect gradual drift patterns. Furthermore, a 'Column-Specific Baseline Validation' was performed by grouping analysis by 'chromatography_stage_order' to distinguish column degradation from detector instability. Third, statistical thresholds were applied to classify failures. CUSUM control limits were derived explicitly for each stage using the mean and standard deviation of the normalized UV series. Linear regression was performed between normalized UV and run number per stage to calculate R^2 values. A run was flagged as a 'Critical Failure' if R^2 was less than 0.99 or if the CUSUM statistic exceeded the calculated limits. These classifications were compiled into a paginated report detailing the specific triggers and metrics for each failure.

4 Results and Discussion

The analysis of the 32 sequential runs revealed distinct patterns of performance across different chromatography stages. As illustrated in Figure 1, the left panel displays the normalized UV signals over time, color-coded by stage, with a rolling mean overlay to highlight gradual drift. The data indicates that while early stages exhibited stable baseline performance, later stages showed a progressive increase in signal deviation. The right panel of Figure 1 tracks the concentration percentages (Conc_B_%), confirming that the observed drift in UV signals correlates with changes in analyte concentration. The stratification by stage order was crucial; it revealed that certain stages experienced more severe degradation than others, suggesting localized column fouling or specific matrix interactions. The statistical evaluation confirmed these visual observations. Runs classified as 'Critical Failures' due to $R^2 \downarrow 0.99$ or CUSUM exceedance were identified, indicating a breakdown in the linear relationship between run time and signal intensity or a significant shift in the mean signal beyond expected control limits. The exclusion of the 65% missing concentration data points ensured that the analysis relied on the most reliable metric (Conc_B_%), preventing data dilution. The normalization by system flow rate effectively removed artifacts caused by pump variability, allowing for a clearer view of true detector and column performance. Overall, the results demonstrate that the proposed methodology successfully distinguished between normal operational variance and critical equipment failures, providing a robust basis for maintenance scheduling and quality assurance.

5 Conclusion

In conclusion, this comprehensive data analysis successfully evaluated the temporal stability of HPLC detector signals and concentration metrics across 32 sequential runs. By implementing a rigorous data cleaning protocol that capped negative values and normalized signals against flow rates, the analysis accurately isolated true drift from operational artifacts. The stratification by chromatography stage order proved effective in distinguishing between gradual column degradation and sudden detector instability. The application of statistical thresholds, including CUSUM control limits and R^2 regression analysis, objectively identified critical equipment failures, ensuring that only runs with significant deviations were flagged. The findings underscore the importance of continuous monitoring and the efficacy of the proposed analytical framework in maintaining product quality and safety. Future work should focus on implementing automated alerts based on these thresholds to enable real-time intervention and further investigate the specific causes of failures in the most affected chromatography stages.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_data.csv')

    # 1. Cap negative cumulative values at 0
    # Calculate negative count BEFORE capping to satisfy feedback
    negative_capped_count = int(((df['volume_ml'] < 0) | (df['UV_1_280_ml'] < 0)).sum())
    df['volume_ml'] = df['volume_ml'].clip(lower=0)
    df['UV_1_280_ml'] = df['UV_1_280_ml'].clip(lower=0)
```

```

# 2. Handle missing Conc_B_%
missing_pct_count = df['Conc_B_%'].isna().sum()

# Identify rows where Conc_B_ml is also missing (imputation impossible)
impossible_impute_mask = df['Conc_B_%'].isna() & df['Conc_B_ml'].isna()
excluded_count = impossible_impute_mask.sum()

# For rows with missing Conc_B_% but present Conc_B_ml, impute Conc_B_ml using
# stage median
stage_groups = df.groupby('chromatography_stage_order')
df_imputed = df.copy()
imputed_count = 0

for stage, group in stage_groups:
    if group['Conc_B_%'].isna().sum() > 0:
        valid_ml = group['Conc_B_ml'].dropna()
        if len(valid_ml) > 0:
            median_val = valid_ml.median()
            mask = group['Conc_B_%'].isna() & group['Conc_B_ml'].notna()
            group.loc[mask, 'Conc_B_ml'] = median_val
            imputed_count += mask.sum()

# Apply imputed values back to main dataframe
df_imputed.loc[df_imputed['Conc_B_%'].isna() & ~df_imputed['Conc_B_ml'].isna(),
               'Conc_B_ml'] = df_imputed.loc[df_imputed['Conc_B_%'].isna() & ~df_imputed
               ['Conc_B_ml'].isna(), 'Conc_B_ml'].median()

# Final exclusion: rows where Conc_B_% is still missing (imputation failed)
df_final = df_imputed.dropna(subset=['Conc_B_%'])
final_excluded = excluded_count

# 3. Calculate statistics on cleaned data
vol_stats = df_final['volume_ml'].describe()
uv_stats = df_final['UV_1_280_mAU'].describe()

# 4. Unique chromatography_stage_order values
unique_stages = df_final['chromatography_stage_order'].nunique()

# Write summary to text file
summary_lines = [
    f"Count of imputed Conc_B_% rows: {imputed_count}",
    f"Count of excluded Conc_B_% rows: {final_excluded}",
    f"Count of capped negative values: {negative_capped_count}",
    f"Volume_ml - Min: {vol_stats['min']}, Max: {vol_stats['max']}, Mean: {
        vol_stats['mean']:.2f}",
    f"UV_1_280_mAU - Min: {uv_stats['min']}, Max: {uv_stats['max']}, Mean: {
        uv_stats['mean']:.2f}",
    f"Unique chromatography_stage_order values: {unique_stages}"
]

with open('/workspace/cleaned_data_summary.txt', 'w') as f:
    f.write('\n'.join(summary_lines))
###

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np

```

```

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/aggregated_run_metrics.csv')

    # Centralized validation step
    required_cols = ['run_no', 'chromatography_stage_order', 'Normalized_UV', '
        Conc_B_%', 'run_mean_UV', 'run_mean_Conc_B']
    missing_cols = [col for col in required_cols if col not in df.columns]
    if missing_cols:
        raise ValueError(f"Missing required columns: {missing_cols}")

    # Filter relevant columns
    df = df[required_cols].copy()

    # Ensure Normalized_UV is numeric and handle missing values
    df['Normalized_UV'] = pd.to_numeric(df['Normalized_UV'], errors='coerce')
    df = df.dropna(subset=['Normalized_UV'])

    # Group by stage
    stages = df['chromatography_stage_order'].unique()

    results = []
    r2_summary = {}
    total_r2_failures = 0
    total_cusum_failures = 0

    for stage in stages:
        # Check if stage has data after filtering
        stage_df = df[df['chromatography_stage_order'] == stage]
        if stage_df.empty:
            continue

        stage_df = stage_df.sort_values('run_no').reset_index(drop=True)

        # 1. CUSUM Limits Calculation (Vectorized)
        mean_uv = stage_df['Normalized_UV'].mean()
        std_uv = stage_df['Normalized_UV'].std()
        if std_uv == 0:
            std_uv = 1e-9

        k = 0.5 * std_uv
        h = 4 * std_uv

        # Pre-allocate arrays
        n = len(stage_df)
        cusum_pos = np.zeros(n)
        cusum_neg = np.zeros(n)

        # Vectorized CUSUM calculation
        deviations = stage_df['Normalized_UV'].values - mean_uv
        cusum_pos = np.maximum(0, np.cumsum(np.maximum(0, deviations - k)))
        cusum_neg = np.minimum(0, np.cumsum(np.minimum(0, deviations + k)))

        # Check CUSUM failures (Vectorized)
        cusum_failures = []
        failure_indices = np.where((cusum_pos > h) | (cusum_neg < -h))[0]
        for idx in failure_indices:
            val = cusum_pos[idx] if cusum_pos[idx] > h else cusum_neg[idx]

```

```

        cusum_failures.append({
            'run_no': stage_df.iloc[idx]['run_no'],
            'stage': stage,
            'trigger_metric': 'CUSUM',
            'cusum_value': val
        })
        total_cusum_failures += 1

# 2. R Calculation (Vectorized)
x = np.arange(n)
y = stage_df['Normalized_UV'].values

slope, intercept, correlation_coefficient, _, _ = np.polyfit(x, y, 1)
r_squared = correlation_coefficient ** 2

r2_summary[stage] = r_squared

# Check R failures
if r_squared < 0.99:
    total_r2_failures += 1
    # Flag the stage itself rather than individual rows
    r2_failures = [{
        'run_no': stage_df.iloc[0]['run_no'], # Representative run
        'stage': stage,
        'trigger_metric': 'R ',
        'r2_value': r_squared
    }]
else:
    r2_failures = []

results.extend(cusum_failures + r2_failures)

# 3. Generate Output
output_lines = []
output_lines.append(f"=== Critical Failure Classification Report ===")
output_lines.append(f"Total critical failures (R < 0.99): {total_r2_failures}")
output_lines.append(f"Total critical failures (CUSUM exceeded): {total_cusum_failures}")
output_lines.append("")

# Summary Statistics
output_lines.append(f"=== R Summary per Stage ===")
for stage, r2 in r2_summary.items():
    output_lines.append(f"Stage {stage}: R = {r2:.6f}")
output_lines.append("")

# Paginated Table
page_size = 20
total_pages = (len(results) + page_size - 1) // page_size

for page in range(1, total_pages + 1):
    start_idx = (page - 1) * page_size
    end_idx = min(page * page_size, len(results))
    page_results = results[start_idx:end_idx]

    output_lines.append(f"--- Page {page} of {total_pages} ---")
    output_lines.append(f"{'RunNo': <10} {'Stage': <15} {'Metric': <10} {'Value': <15}")

```

```

output_lines.append("-" * 50)

for row in page_results:
    val_str = f"{row['cusum_value']:.4f}" if row['trigger_metric'] == '
    CUSUM' else f"{row['r2_value']:.6f}"
    output_lines.append(f"{row['run_no']:<10}_{row['stage']:<15}_{row['
    trigger_metric']:<10}_{val_str:<15}")

output_lines.append("")

# Write to file
with open('/workspace/failure_classification.txt', 'w') as f:
    f.write('\n'.join(output_lines))
###

```

Listing 2: Code_3